

Hybrid Scientific Machine Learning Framework for Performance Prediction of Helical Coil Heat Exchangers

Suyash Dixit
Dept. of Mechanical
Engineering
MIT Academy of
Engineering
Pune, India

Hrushant Motinge
Dept. of Computer
Engineering
MIT Academy of
Engineering
Pune, India

Aryan Tiwari
Dept. of Computer
Engineering
MIT Academy of
Engineering
Pune, India

Vedant Salvi
Dept. of Computer
Engineering
MIT Academy of
Engineering
Pune, India

Dr. Diptee Ghusse
School of Computer Engineering
MIT Academy of Engineering
Pune, India
diptee.ghusse@mitaoe.ac.in

Mrs. Sharmila Kharat
School of Computer Engineering
MIT Academy of Engineering
Pune, India
sharmila.kharat@mitaoe.ac.in

Dr. Pramod Kothmire
Associate Professor
Department of Mechanical
Engineering
MIT Academy of Engineering

Abstract— This paper discusses a method for predicting the performance of Helical Coil Heat Exchangers. We made a framework that uses Scientific Machine Learning to figure out the hydraulic performance of Helical Coil Heat Exchangers. We tried two neural network architectures: one that just uses the data we have and another that also uses rules from physics to make sure the answers make sense. We want to know three things about Helical Coil Heat Exchangers: how well they can exchange heat how much pressure is lost and how well they can transfer heat overall. To do this we need to know things about the heat exchanger like its shape and how the fluid is moving. We also calculate some numbers, like the Reynolds number and the Dean number to make sure everything is consistent. Comparative evaluation demonstrates that the PINN architecture outperforms the data-driven MLP in terms of generalization and adherence to governing physics. A deployable digital twin is implemented using Streamlit allowing real-time inference. Results confirm the utility of physics-informed approaches in modeling for compact heat exchanger systems.

Keywords— coil heat exchanger, Physics-informed neural networks, Scientific machine learning, Digital twin, Heat transfer, Pressure drop prediction, Deep learning surrogate model

I. INTRODUCTION

Helical Coil Heat Exchangers are used a lot in industries that need to transfer heat, like refrigeration and chemical plants. They are good at transferring heat because of their shape, which creates swirls in the fluid that help to move heat. However it is hard to understand how they work because the fluid and heat are interacting in ways.

Usually we use methods to design Helical Coil Heat Exchangers. These methods do not work well when things get

complicated. We can use computer simulations to get an answer but these simulations take a lot of time and computer power. We need a way to quickly and accurately predict how well Helical Coil Heat Exchangers will perform.

The rise of data-driven modeling provides a different option. Artificial neural networks have shown they can predict heat exchanger performance by learning non-linear relationships from operational data. However, data-driven methods have a key limitation. They do not ensure physical consistency, and their extrapolation can lead to unrealistic results, like negative heat transfer coefficients or efficiency values over one.

Physics-Informed Neural Networks, introduced by Raissi et al., provide a solution to this limitation by incorporating governing physical equations directly into the neural network training loss. This method limits the solution space to achievable outputs. It also improves the generalization and reliability of predictions.

This work proposes a SciMqL framework that combines both paradigms. An MLP baseline is trained on operational data followed by a PINN variant that incorporates physics residuals corresponding to pressure drop, heat transfer and overall heat transfer coefficient consistency into its loss function. Both models are evaluated on accuracy metrics. Deployed as a real-time digital twin via Streamlit.

II. LITERATURE REVIEW

Research in data-driven modeling has grown substantially over the past decade. Early neural network applications to heat exchangers focused on shell-and-tube configurations establishing that feedforward networks could approximate NTU-based effectiveness predictions with accuracy. More recent efforts extended these approaches to heat exchangers and helical coil geometries exploiting the richer dataset structures afforded by CFD-generated training samples.

Simultaneously Nusselt number correlations specific to helical coils. Including the Mori-Nakayama and Dravid correlations. Have served as the dominant physics reference

for tube-side heat transfer in curved passages. These correlations account for the Dean number effect. Remain the basis for most analytical models employed in industry.

The concept of physics-informed machine learning gained prominence with the work of Raissi et al. which demonstrated that partial differential equations could be incorporated as constraints within the training loss of deep neural networks. Since then this approach has been applied to flow, heat conduction and structural mechanics problems with demonstrated improvements in data efficiency and physical plausibility of predictions.

Within the heat exchanger domain recent studies have explored PINN-based modeling for heat conduction and conjugate heat transfer problems. However, the prediction of the performance of Helical Coil Heat Exchangers, such as fluid flow and heat transfer using PINNs has not been studied much. That's what we are trying to discover here.

III. PROBLEM FORMULATION AND SYSTEM INPUTS

Our goal is to create a model that can take some inputs about how something works and its design and then give us three key outputs:

- * the efficiency of a heat exchanger
- * how much pressure is lost
- *. Its ability to transfer heat.

These inputs are classified into three types as shown below in Table I

TABLE I. Input Parameter Summary

Category	Parameter	Description
Hot Side	T_{hot}, m_{hot}	Inlet temperature (°C), mass flow rate (kg/s)
	ρ, μ, Cp, k, Pr	Density, viscosity, specific heat, conductivity, Prandtl number
Cold Side	T_{cold}, m_{cold}	Inlet temperature (°C), mass flow rate (kg/s)
	ρ, μ, Cp, k, Pr	Density, viscosity, specific heat, conductivity, Prandtl number
Geometry	d, D_{coil}, p, N	Tube diameter, coil diameter, pitch, number of turns
	D_{shell}, t	Shell diameter (m), simulation time (s)

One of the main design choices of this framework is that the Reynolds number (Re) and Dean number (De) are not accepted as user inputs. Instead, they are computed internally from the mass flow rate, fluid properties, and geometry using the following sequential chain:

$$v = m / (\rho \cdot A) \quad (1)$$

$$Re = (\rho \cdot v \cdot d) / \mu \quad (2)$$

$$De = Re \cdot \sqrt{(d / D_{coil})} \quad (3)$$

This ensures physical self-consistency throughout the prediction pipeline. Added the option to accept Re or De externally, so users can specify values not consistent with the mass flow rates, identified as a source of erroneous predictions during initial development.

Additionally, the coil length is derived from the helix geometry as given below:

$$L = N \cdot \sqrt{(\pi D_{coil})^2 + p^2} \quad (4)$$

where N is the number of turns, D_{coil} is the coil diameter, and p is the pitch per turn. This closed form expression takes into account the circumferential and axial components of the helical path.

IV. GOVERNING PHYSICS EQUATIONS

The PINN loss function contains physical constraints from well-known relations of heat transfer and fluid mechanics for internal helical flow.

A. Heat Capacity Rates

$$C_h = m_h \cdot Cp_h, \quad C_c = m_c \cdot Cp_c \quad (5)$$

$$C_{min} = \min(C_h, C_c), \quad C_r = C_{min} / C_{max} \quad (6)$$

B. Convection Coefficients

The convective heat transfer coefficient on each side is obtained from Nusselt number correlations for flow in a helical tube:

$$h = (Nu \cdot k) / d \quad (7)$$

C. Overall Heat Transfer Coefficient

Neglecting fouling resistance and tube wall conduction resistance (thin-wall assumption):

$$U = 1 / ((1/h_h) + (1/h_c)) \quad (8)$$

D. Pressure Drop

The Darcy-Weisbach equation is used over the length of the helical coil:

$$\Delta P = f \cdot (L / d) \cdot (\rho v^2 / 2) \quad (9)$$

Here f is a measure of the Fanning friction factor. It is modified for helical geometry. This means it takes into account the resistance that comes from the centrifugal acceleration of the Dean vortices

E. Heat Exchanger Effectiveness

Under the NTU- ϵ formulation, effectiveness is expressed as:

$$\epsilon = Q_{actual} / Q_{max} \quad (10)$$

where $Q_{max} = C_{min} \cdot (T_{h,in} - T_{c,in})$ represents the thermodynamic maximum heat transfer achievable.

V. METHODOLOGY

A. Data Generation and Preprocessing

The synthesis of the data set involved sampling the input parameters over physically plausible ranges through the application of Latin Hypercube Sampling (LHS). The outputs were calculated using the mathematical relationships in Section IV. This generated data set was further split up into two data sets -training (80%) and testing (20%).

All input features are zero mean, unit variance scaled by StandardScaler, and output targets are scaled separately in

order to normalize different scale between , P and U. Separate scalars for inputs and outputs were serialized and bundled with the deployed model to ensure consistent inference.

B. MLP Architecture

The base model, used in this thesis, is an feedforward Multilayer Perceptron (MLP), built using PyTorch. The model takes 28 input features, which include 20 original input parameters along with 8 additional features generated internally, and predicts 3 output values.

The network architecture consists of four hidden layers containing 256, 128, 64, and 32 neurons respectively. To improve training stability and model performance, each hidden layer is followed by batch normalization and a ReLU activation function. We also used something called dropout to help the model work better. This means that sometimes the model will ignore some of the information it gets and this helps prevent it from getting too good at the training data. We set the probability of this happening to 0.2.

For making the model work we used the Adam optimizer. This is a way of helping the model learn from its mistakes. We started with a learning rate of 1 times 10 to the power of -3.

We also used something called ReduceLROnPlateau to help the model learn. This means that if the model stops getting better the learning rate will go down automatically.

The model was trained using the Mean Squared Error loss function. This is a way of measuring how good the model is at making predictions. It looks at how different the model's predictions are from the real answers and it does this for all three things the model is trying to predict at the same time.

The loss function is calculated like this: L_{MLP} = the average of the sum of the squared differences, between what the model predicts and what really happens. This is written as $L_{MLP} = (1/N) \sum ||\hat{y}_i - Y_i||^2$ (11)

C. PINN Architecture and Physics Loss

The PINN shares the identical network architecture as the MLP. The distinction lies solely in the training objective. In addition to the standard data-fitting loss, the PINN incorporates physics residuals computed from the network predictions:

Pressure drop residual — the predicted ΔP is compared against the value recomputed from (9) using predicted velocity and friction factor from the predicted Re.

U-value consistency residual — the predicted U is compared against the value recomputed from (8) using convective coefficients derived from the predicted Nu correlations.

The total PINN loss is made up of a things combined together. L_{PINN} is calculated as λ_{data} times L_{data} plus $\lambda_{\Delta P}$ times $R_{\Delta P}$ plus λ_U times R_U . The factors responsible for controlling the relative contributions of each term are λ_{data} , $\lambda_{\Delta P}$ and λ_U . They are like adjustable parameters that determine the performance of the PINN. We attempted to use another parameter known as effectiveness residual but were not successful because it caused the predicted values of effectiveness to be very similar to each other due to making the solution too optimal. Training of the PINN takes place in two phases. In the first phase, we used L_{data} while in the

second phase, we introduced the physics residuals gradually so as to avoid conflicts between them initially.

D. Tabular ResNet

ResNet model was also developed to enhance learning performance on structured thermal-fluid datasets. During experimentation, deeper neural networks exhibited unstable learning behavior because gradient values became extremely small after a few layers. Residual learning reduced this effect and improved optimization stability.

The network did not learn only a single mapping but instead learned residual mappings between layers. The residual mapping can be written as follows:

$$y = F(x, W) + x$$

where x represents the input feature vector whereas $F(x, W)$ represents the nonlinear transformation mapping learned by the network. The variable W contains trainable weight parameters whereas y represents the output obtained after passing the input vector through the residual mapping.

Hidden layer transformations within residual blocks were computed as follows:

$$F(x) = \sigma(W_2 \cdot \sigma(W_1 x + b_1) + b_2)$$

Here, W_1 and W_2 represent weight matrices used during training, while b_1 and b_2 represent bias values. The activation function is represented by σ .

ReLU activation was selected because it improved nonlinear learning behavior during experimentation.

$$ReLU(x) = \max(0, x)$$

Training showed stronger relationships learned between Reynolds number, Dean number, pressure drop, and heat transfer parameters using the proposed architecture compared to the baseline MLP model.

Another observation during training was that residual learning reduced degradation problems in deeper layers and improved convergence stability.

Prediction error was calculated using Mean Squared Error:

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

In this expression, y_i stands for the actual experimental value while $\hat{y}_i$ stands for the predicted output generated by the model. The parameter N represents the total number of samples used during training.

E. Physics-Constrained Tabular Transformer

A transformer network architecture was employed in the course of experimentation to learn complex interactions between features in thermofluidic datasets.

As opposed to traditional feedforward networks, transformer architecture employs

an attentional mechanism to discover relationships among multiple features simultaneously. In practice, parameters like Reynolds number, Dean number, curvature ratio, pressure drop, and heat transfer coefficient interact significantly.

Attention mechanism proved helpful in learning such relationships.

The self-attention mechanism used within transformer architecture was defined as:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Here, Q corresponds to the Query matrix, K is the Key matrix, and V stands for the Value matrix. The term d_k stands for the dimensional size of the key vectors.

The Query, Key, and Value matrices were generated from the input dataset using trainable weight matrices:

$$\begin{aligned} Q &= XW_Q \\ K &= XW_K \\ V &= XW_V \end{aligned}$$

In these equations, X stands for the normalized input dataset while W_Q , W_K , and W_V represent trainable weight matrices used during optimization.

F. Evaluation Protocol

The constructed models were analyzed using traditional statistics and

parity plots based on the held-out test data. While testing, comparison

was performed between prediction accuracy and stability using Root

Mean Squared Error (RMSE), Mean Absolute Error (MAE), and coefficient

determination (R^2).

Root Mean Squared Error was computed by:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

where y_i represents the actual experimental value, $\hat{y}_i$ represents the predicted value, and N represents the total number of testing samples.

Mean Absolute Error was computed as:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

The coefficient of determination was calculated using:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

where $\bar{y}$ stands for the mean of actual experimental values.

Parity plots were also obtained during testing to compare predicted outputs with experimental observations. A prediction was slightly considered accurate when the plotted points remained close to the diagonal parity line.

For the transformer-based architecture, the self-attention mechanism used inside the model was calculated as:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Here, Q represents the Query matrix, K represents the Key matrix, and V represents the Value matrix. The parameter d_k represents the dimensional size of the key vectors used during attention computation.

VI. RESULTS AND DISCUSSION

Table II shows the relative performances of MLP and PINN on test data for two main outputs: efficiency and pressure drop.

TABLE II. Comparative Model Performance on Test Set

Model	RMSE (ε)	RMSE (ΔP)	R ² (ε)	R ² (ΔP)
MLP	0.0182	1.243	0.9713	0.9601
PINN	0.0165	1.108	0.9784	0.9712

The RMSE for effectiveness drops from 0.0182 for MLP to 0.0165 for PINN, a reduction of 9.3%.

The RMSE for pressure drop drops from 1.243 to 1.108, a reduction of 10.9%.

We see improvement in R value which means it does seem that incorporating physical constraints actually helped PINN predict values. The RMSE for effectiveness and pressure drop is better in the PINN.

Parity plots show good prediction within a range that is very close to the identity line for both models over a large part of the output space. However, at higher values of P the MLP is slightly and systematically underpredicting. This is greatly alleviated in the PINN which implies the physical constraints decrease over-fitting to dense areas near the center of the training distribution.

Internal computation of the Reynolds and Dean numbers were key to enforcing physical realism. These were originally user inputs to the model, which produced systematic errors in the pressure drop when values provided by the user were contradictory to stated mass flow rate values. Internally

computing these values via the velocity-Reynolds-Dean chain resolved this type of error.

The effectiveness residual was removed from the PINN loss following observation of output collapse. This is attributed to the strong coupling between effectiveness, heat capacity ratios, and temperature differences - multiple competing constraints simultaneously attempting to minimize residuals on a bounded output variable produced unstable training dynamics. Removing the effectiveness residual while retaining ΔP and U residuals yielded stable convergence.

VII. DIGITAL TWIN DEPLOYMENT

We created a web application using Streamlit to serve as the interface, for our twin.

The application uses -trained model weights (mlp_model.pth, pinn_model.pth) and some configuration files (x_scaler.pkl, y_scaler.pkl model_config.json).

Here is how the application works:

- * The user inputs 20 parameters through a sidebar.
- * The application then calculates some information internally.
- * Our MLP and PINN models make predictions on their own.
- * The application shows both predictions and an average of the two. The user can compare them.

The outputs are visible for the user. Is there consensus among the models?

The DIGITAL TWIN DEPLOYMENT begins the features shown here, for ease of use.

The application lets the user to easily interact with the DIGITAL TWIN DEPLOYMENT.

The averaged MLP and PINN predictions using an ensemble method reduces the error when one model is performing poorly near the boundary of the domain of input. This approach leverages an existing ensemble learning method without further computational cost.

A engineering problem faced during the deployment was obtaining negative U and P , both impossible physical quantities. A mismatch in the architecture parameters definition, between the deployment script and the training process resulted in the issue, and it was resolved by loading architecture definition from the configuration file on application start-up. This prevents issues related to the hardcoding of architecture definitions. The configuration file enables definition of the architecture without having it in the application.

VIII. CONCLUSION AND FUTURE WORK

This paper has presented a hybrid SciML framework to predict performance metrics for Helical Coil Heat Exchangers. This model has combined standard data driven MLP model and the PI-NN which embeds fundamental thermal and hydraulic constraints of the problem. The system is capable of predicting heat exchanger effectiveness, pressure drop and the heat transfer coefficient based on 20 thermofluid and geometric inputs; key intermediate computed values are

generated internally to preserve physical consistencies of the model outputs.

In this work, the proposed PINN model performance and accuracy are better and physically sound in comparison with that of a pure MLP based model with reduced RMSE of approximately 9-11% on the main output parameters. A Streamlit digital twin of the models' predictive power has been developed and is interactive in real-time to make inferences.

A number of decisions which are applicable to general SciML practices are also discussed herein: i) Removing the effectiveness physics residual so that it does not impose conflicting information onto a bounded output value. ii) Allowing the model to compute Re and De internally rather than passing them as user-defined inputs to ensure logical consistencies. iii) Employing a two phase PINN training approach to overcome the issues of loss conflicts.

Further investigation and extension will include additional model architectures such as; Residual Network (ResNet), TabNet, Autoencoder-Regressor hybrid and XGBoost for gradient boosted comparisons. Investigations for prediction intervals (e.g., through Monte Carlo Dropout or Deep Ensembles) will also be carried out. The developed system will be further validated against physical experimental data gathered from a laboratory scale HCHE test rig.

IX. REFERENCES

- [1] S. Haykin, Neural Networks and Learning Machines, 3rd ed. Pearson Prentice Hall, Upper Saddle River, NJ, USA, 2009.
- [2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational Physics*, vol. 378, pp. 686-707, 2019.
- [3] A. P. Fraas, Heat Exchanger Design, 2nd ed. John Wiley & Sons, New York, NY, USA, 1989.
- [4] F. P. Incropera, D. P. DeWitt, T. L. Bergman, and A. S. Lavine, Fundamentals of Heat and Mass Transfer, 7th ed. John Wiley & Sons, Hoboken, NJ, USA, 2011.
- [5] Y. A. Çengel and A. J. Ghajar, Heat and Mass Transfer: Fundamentals and Applications, 5th ed. McGraw-Hill Education, New York, NY, USA, 2015.
- [6] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, Cambridge, MA, USA, 2016.
- [7] S. Mori and W. Nakayama, Study on forced convective heat transfer in curved pipes, part I: Laminar region, *International Journal of Heat and Mass Transfer*, vol. 8, no. 1, pp. 67-82, 1965.
- [8] G. Cybenko, Approximation by superpositions of a sigmoidal function, *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303-314, 1989.
- [9] E. Haghighi and S. Khodadadi, Machine learning models for prediction of heat exchanger performance: A comparative study, *Applied Thermal Engineering*, vol. 174, 115351, 2020.
- [10] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.